UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CIÊNCIA DA COMPUTAÇÃO

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Florianópolis
2026

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina como requisito para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Antônio Augusto Fröhlich, Phd.
Coorientador: Matheus Wagner, Me.

Florianópolis
2026

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de Bacharel em Ciência da Computação e aprovado em sua forma final pelo curso de Graduação em Ciência da Computação.

Florianópolis, _____ de _____ de 2026.

Banca Examinadora:

Prof. Antônio Augusto Fröhlich, Phd.
Orientador
Universidade Federal de Santa Catarina

Matheus Wagner, Me.
Coorientador
Universidade Federal de Santa Catarina

José Luis Conradi Hoffmann, Phd.
Avaliador
Universidade Federal de Santa Catarina

RESUMO

A convergência entre robótica móvel e Internet das Coisas é central para a implementação dos Sistemas Ciber-Físicos. O *Robot Operating System 2* (ROS 2) consolidou-se como o padrão no desenvolvimento robótico e adotou o *Data Distribution Service* (DDS) como *middleware* de comunicação, garantindo entrega eficiente de mensagens. As mensagens nativas do ROS, porém, transmitem dados brutos de sensores sem o contexto físico e temporal necessário para sistemas de tomada de decisão autônoma. O conceito de *SmartData*, proposto por Fröhlich (2018), responde a essa lacuna ao encapsular cada leitura sensorial em um envelope que carrega, inseparavelmente do valor, sua unidade no Sistema Internacional, a origem espaço-temporal do dado e seu prazo de validade. Este trabalho projeta, implementa e avalia um *Gateway* semântico que opera como nó ROS 2 e traduz mensagens de sensores robóticos para o formato SmartData. A solução adota uma estratégia de geração de código em tempo de compilação a partir de um arquivo declarativo: um descritor YAML enumera os tópicos a converter e suas regras de decomposição, e um gerador em Python emite os quatro *headers* C++ que realizam a tradução. O pacote SmartData resultante é difundido em um barramento UDP multicast binário de 72 bytes, compatível com transdutores SmartData externos. A validação empírica em ambiente conteinerizado mediu uma latência de *marshaling* de $0{,}8\,\mu\mathrm{s}$ no 95º percentil para uma mensagem de varredura LiDAR de 100 feixes, valor três ordens de grandeza abaixo do piso de latência do ROS 2 para cargas pequenas reportado na literatura, entre $1\,\mathrm{ms}$ e $3\,\mathrm{ms}$. Os resultados sustentam a viabilidade de integrar robôs ROS 2 a ecossistemas SmartData sem comprometer o desempenho do laço de controle.

Palavras-chave: ROS 2; SmartData; Sistemas Ciber-Físicos; Interoperabilidade Semântica; Tradução Semântica; UDP Multicast.

SUMÁRIO

1 INTRODUÇÃO 6
1.1 Problema de Pesquisa 6
1.2 Hipótese 7
1.3 Justificativa 7
1.4 Objetivos 7
1.4.1 Objetivo Geral 7
1.4.2 Objetivos Específicos 8
1.4.3 Escopo e Limites 8

2 FUNDAMENTAÇÃO TEÓRICA 9
2.1 Sistemas Ciber-Físicos 9
2.2 Robot Operating System 2 9
2.2.1 Modelo de Comunicação 10
2.2.2 Implementação por Camadas 10
2.2.3 Caracterização de Desempenho 10
2.2.4 Árvore de Transformações 10
2.3 O Conceito de SmartData 11
2.3.1 Estrutura do Envelope 11
2.3.2 Origem Espaço-Temporal e Validade 11
2.3.3 Endereçamento e Discriminação por Faixas de Dispositivo 12

3 TRABALHOS RELACIONADOS 13
3.1 Desempenho de Middleware Robótico 13
3.2 Interoperabilidade Semântica em IoT 13
3.3 Implementações de SmartData 14

4 DESIGN DO SISTEMA 15
4.1 Visão Geral 15
4.2 Linguagem Declarativa de Sensores 16
4.3 Geração de Código em Tempo de Compilação 16
4.4 Discriminação por Faixas de Dispositivo 18
4.5 Transportes do SmartData 18
4.5.1 Barramento UDP Multicast 18
4.5.2 Espelho em Tópico ROS 18

5 IMPLEMENTAÇÃO 20
5.1 Estrutura do Pacote 20

5.2 Codificação da Unidade SI . . . 21
5.3 Envelope Binário e Barramento . . . 21
5.4 Caminho ROS para SmartData . . . 21
5.5 Caminho SmartData para ROS . . . 22
5.6 Geração de Código . . . 22

6 PLANO E EXECUÇÃO DE EXPERIMENTOS . . . 23
6.1 Ambiente Experimental . . . 23
6.2 Cenários . . . 23
6.2.1 Cenário Sintético . . . 23
6.2.2 Cenário Gazebo . . . 24
6.3 Métricas e Procedimentos . . . 24
6.3.1 Latência de *Marshaling* . . . 24
6.3.2 Vazão Sustentada . . . 24
6.4 Resultados . . . 24
6.4.1 Latência . . . 24
6.4.2 Vazão . . . 25
6.4.3 Síntese . . . 25
6.5 Discussão . . . 25

7 CONSIDERAÇÕES FINAIS . . . 26
7.1 Trabalhos Futuros . . . 26

REFERÊNCIAS . . . 27

1 INTRODUÇÃO

A evolução dos Sistemas Ciber-Físicos (*Cyber-Physical Systems*, CPS) e a consolidação da Internet das Coisas transformaram a forma como máquinas interagem com o mundo físico. Como argumenta Lee (2008), a essência dos CPS reside na orquestração integrada de computação, comunicação e controle, onde a dinâmica dos processos físicos exige o mesmo rigor formal que as computações lógicas. Nesse cenário, a robótica móvel deixou de operar como sistema isolado e tornou-se um componente ativo em ambientes inteligentes, integrada a infraestruturas de instrumentação, controle e analítica que extrapolam os limites do robô.

No domínio do desenvolvimento robótico, o *Robot Operating System* (ROS) estabeleceu-se como o padrão de fato para a construção de sistemas complexos. O ROS 2 introduziu uma mudança arquitetural significativa ao substituir seu transporte próprio pelo *Data Distribution Service* (DDS), um padrão da OMG voltado a sistemas distribuídos de tempo real (MARUYAMA et al., 2016; MACENSKI et al., 2022). Paralelamente, no contexto de IoT industrial e cidades inteligentes, o conceito de *SmartData*, proposto por Fröhlich (2018), oferece uma abordagem orientada a dados que enriquece a informação bruta com metadados de semântica, localização e validade temporal, tornando cada leitura sensorial um objeto autocontido.

Apesar de coexistirem no mesmo nicho de aplicações ciber-físicas, os dois modelos de dados operam em níveis distintos de abstração. As mensagens nativas do ROS são projetadas para a eficiência do laço de controle e expõem campos numéricos ou estruturas pouco anotadas, das quais o significado semântico, como, unidade física, validade temporal e referencial espaço-temporal, é convenção do projeto, não dependente da mensagem. O SmartData, por sua vez, encapsula essas informações no próprio envelope, mas é difundido em um barramento próprio incompatível com o ferramental ROS. Serrano, Hauswirth e Soldatos (2015) caracterizam esse isolamento como um dos problemas centrais à interoperabilidade na Internet das Coisas, padrão que se reproduz na fronteira entre robótica e instrumentação ciber-física e motiva a tradução proposta neste trabalho.

1.1 PROBLEMA DE PESQUISA

A motivação deste trabalho está na ausência de um mecanismo direto e eficiente de tradução entre as mensagens operacionais do ROS 2 e o formato semanticamente enriquecido do SmartData. *Gateways* de aplicação genéricos existem, mas tipicamente introduzem latência incompatível com laços de controle robóticos ou perdem metadados durante a serialização. Formula-se, portanto, a seguinte questão de pesquisa: como

viabilizar a interoperabilidade semântica entre sistemas robóticos baseados em ROS 2 e plataformas de CPS orientadas a SmartData, mantendo os requisitos de tempo real e baixo *overhead* de processamento?

1.2 HIPÓTESE

Este trabalho conjectura que é possível desenvolver um componente operando nativamente no ecossistema ROS 2 capaz de interceptar, enriquecer e retransmitir mensagens de sensores no formato SmartData, com uma latência de *marshaling* pequena o suficiente para não afetar o funcionamento dos demais subsistemas. Assume-se que, ao realizar a tradução dentro do próprio nó ROS, aproveitando as proprias informações do ecossistema para preencher os metadados referentes de uma mensagem SmartData, elimina-se a necessidade de *brokers* externos complexos, preservando a integridade semântica dos dados.

1.3 JUSTIFICATIVA

A relevância desta pesquisa decorre da transição da arquitetura de sistemas robóticos para ecossistemas conectados e centrados em dados. A maturidade alcançada pelo ROS 2 (MACENSKI et al., 2022) resolveu problemas críticos de descoberta, escalabilidade e qualidade de serviço no transporte, mas não endereçou nativamente a compreensão semântica da informação: o *middleware* garante a entrega da mensagem, não que o receptor interprete inequivocamente a unidade física, o referencial de coordenadas ou a validade temporal do dado sem configuração prévia manual.

A lacuna semântica representa risco para a segurança e a confiabilidade de sistemas autônomos heterogêneos. Prestes et al. (2013) apontam que a falta de uma ontologia padronizada e acoplada aos dados em tempo de execução é causa frequente de falhas na composição de robôs de fabricantes ou módulos distintos. O SmartData mitiga esse risco ao acoplar inseparavelmente o valor numérico aos seus metadados de semântica, espaço e tempo (FRÖHLICH, 2018). Do ponto de vista de desempenho, executar a tradução dentro do próprio processo ROS, e não em um processo externo encadeado por mais uma travessia de serialização, preserva a viabilidade do caminho para laços de controle em tempo real (KRONAUER et al., 2021).

1.4 OBJETIVOS

1.4.1 Objetivo Geral

Projetar, implementar e validar experimentalmente um *Gateway* semântico para ROS 2 capaz de interceptar mensagens de sensores padrão e retransmiti-las em tempo

real no formato SmartData, preservando a integridade semântica, espacial e temporal dos dados originais.

1.4.2 Objetivos Específicos

1. Definir um modelo formal de correspondência entre mensagens de sensores ROS e a estrutura tipada do SmartData, expressível por meio de um arquivo declarativo.
2. Implementar o *Gateway* como nó nativo do ROS 2, aproveitando a árvore de transformações e o relógio do próprio ecossistema para enriquecer cada amostra com sua origem espaço-temporal.
3. Difundir o SmartData produzido em um barramento binário UDP multicast compatível com transdutores SmartData nativos, oferecendo como caminho secundário um espelhamento em tópico ROS para instrumentação e depuração no próprio ecossistema.
4. Caracterizar empiricamente a latência de *marshaling* e a vazão sustentada do tradutor, a partir exclusivamente do barramento SmartData.

1.4.3 Escopo e Limites

A pesquisa delimita-se à camada de *software* de aplicação sobre ROS 2 Humble em Linux, tratando o DDS subjacente apenas como o transporte padrão do ROS, sem propor modificações ao *middleware*. A validação restringe-se a cenários simulados (gerador sintético e Gazebo) para garantir a reprodutibilidade. Mecanismos criptográficos previstos pelo SmartData estão fora do escopo deste trabalho, que se concentra na estrutura de dados e na caracterização de desempenho.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os três pilares conceituais sobre os quais o *Gateway* foi construído: Sistemas Ciber-Físicos como contexto da demanda por dados semanticamente ricos, o ROS 2 como plataforma robótica baseada em DDS, e o conceito de SmartData como construção de envelope para dados de sensores. Cada seção descreve as definições, propriedades e limitações que motivaram decisões de projeto adotadas nos capítulos posteriores.

2.1 SISTEMAS CIBER-FÍSICOS

Sistemas Ciber-Físicos são, na definição de Lee (2008), integrações de computação e processos físicos nas quais computadores embarcados e redes monitoram e controlam o mundo físico, comumente em laços de realimentação. O autor argumenta que a passagem do tempo, no mundo físico, é inevitável e a concorrência é intrínseca; ambas as propriedades, no entanto, não são tratadas pelas abstrações usuais, herdadas da computação de propósito geral. As consequências práticas incluem a dificuldade de raciocinar formalmente sobre latência, determinismo e ordenação em sistemas que mediam controle físico.

A demanda por confiabilidade em CPS é qualitativamente distinta da encontrada em *software* convencional. Em uma aplicação de processamento de texto, uma falha pode produzir desconforto, já em um sistema de controle automotivo, ela pode produzir danos físicos. Lee enfatiza que as decisões de projeto em CPS devem incorporar explicitamente o tempo e a concorrência como cidadãos de primeira classe, e não como propriedades emergentes da implementação. Esse argumento da base à adoção de mecanismos como o *Data Distribution Service* pelo ROS 2 e ao enriquecimento temporal dos pacotes SmartData no presente trabalho.

2.2 ROBOT OPERATING SYSTEM 2

O ROS é um conjunto de bibliotecas e ferramentas para o desenvolvimento de sistemas robóticos; o ROS 2 é seu sucessor, redesenhado para abordar limitações herdadas da primeira versão em segurança, tempo real e operação em redes não ideais (MACENSKI et al., 2022). A diferença arquitetural mais relevante para este trabalho é a substituição do transporte original do ROS 1 pelo DDS, um padrão da *Object Management Group* para comunicação dirigida a dados, baseado no modelo publish/subscribe com descoberta distribuída e configurações de qualidade de serviço (QoS) (MARUYAMA et al., 2016).

2.2.1 Modelo de Comunicação

No modelo de comunicação do ROS 2, processos (*nodes*) publicam e assinam tópicos identificados por nome e tipo de mensagem. Cada tipo de mensagem é definido em uma linguagem de descrição de interface, a *Interface Definition Language* (IDL) usada pelo DDS, e compilado a estruturas de dados nativas em C++ e Python. O publicador serializa uma instância da mensagem segundo o IDL e a entrega à camada DDS, que a roteia para todos os assinantes interessados de acordo com a política de QoS associada ao tópico.

A qualidade de serviço é configurável por tópico e abrange, entre outros, profundidade do histórico, confiabilidade (*reliable* ou *best-effort*), durabilidade e prazos. O ROS 2 oferece perfis nomeados como `SensorDataQoS`, otimizado para dados sensoriais de alta frequência, e `SystemDefaultsQoS`, que herda a configuração padrão do *middleware*.

2.2.2 Implementação por Camadas

A pilha do ROS 2 segue uma estratificação clara (MACENSKI et al., 2022; MARUYAMA et al., 2016). Bibliotecas cliente como `rclcpp` expõem o modelo de *publish*/*subscribe* ao programador e delegam a comunicação a um *middleware interface module* (`rmw`), que por sua vez encapsula uma implementação concreta de DDS, como Fast DDS, Cyclone DDS ou Connext. A separação entre `rmw` e a implementação de DDS permite que o mesmo código de aplicação rode sobre fornecedores distintos.

2.2.3 Caracterização de Desempenho

A latência de comunicação do ROS 2 foi caracterizada em diversos estudos. Maruyama et al. (2016) mostraram que a sobrecarga da pilha do ROS 2 sobre o DDS bruto varia significativamente conforme o cenário, com valores da ordem de centenas de microssegundos para mensagens pequenas em processos distintos. Kronauer et al. (2021) estenderam essa análise para sistemas com múltiplos nós e mostraram que o ROS 2 pode introduzir até 50% de *overhead* sobre o DDS bruto para mensagens pequenas, com valores típicos no 95º percentil entre uma e três milissegundos para cargas de $128\,\mathrm{B}$ a $500\,\mathrm{kB}$ em configurações representativas. Esses dados são a referência mais diretamente comparável ao *overhead* adicional que um tradutor para SmartData deve evitar reintroduzir.

2.2.4 Árvore de Transformações

O subsistema `tf2` do ROS mantém uma árvore de transformações geométricas entre quadros de referência, indexada no tempo. Cada par de quadros é vinculado por uma transformação contínua, e consultas sobre essa árvore retornam, para um

instante específico, a transformação resultante da composição ao longo de um caminho da árvore. Essa estrutura é a fonte canônica para preencher os campos de origem espacial do SmartData a partir do `frame_id` declarado no cabeçalho das mensagens de sensor.

2.3 O CONCEITO DE SMARTDATA

O SmartData foi proposto por Fröhlich (2018) como a abstração primária usada por aplicações para interagir com o mundo físico em uma rede de sensores e atuadores. Trata-se de um dado enriquecido com metadados suficientes para ser autocontido quanto à semântica, à localização espacial, ao instante de produção e à confiabilidade. Conceitualmente, ler uma instância de SmartData implica amostrar o transdutor associado, e escrever em uma instância define um novo *setpoint* para um atuador.

2.3.1 Estrutura do Envelope

Cada envelope SmartData carrega, além do valor numérico, uma unidade codificada como inteiro de 32 bits que descreve a dimensão SI da grandeza sensoriada, as coordenadas (x, y, z) da origem espacial, um *timestamp* de alta resolução, um prazo de validade (*expiry*) relativo ao instante de produção e um código de autenticação. A unidade é codificada com um bit de tipo (físico/digital), dois bits para o tipo numérico, dois bits para um modificador (direto, inverso, logarítmico) e nove campos de três bits para os expoentes das sete dimensões fundamentais do SI mais radiano e esterradiano, conforme ilustrado na Figura 1 do trabalho original.

Essa codificação produz, por exemplo, a constante `METER` com expoente $+1$ na dimensão de comprimento e zero nas demais; `METER_PER_SEC_SQ` com expoentes $+1$ em comprimento e -2 em tempo, descrevendo aceleração; e `VOLT` como composto das dimensões de massa, comprimento, tempo e corrente segundo a definição formal. Portanto, a semântica é intrínseca ao identificador de unidade, sem necessidade de tabelas de tradução externas.

2.3.2 Origem Espaço-Temporal e Validade

O *timestamp* de origem é expresso em microssegundos contados a partir de uma época absoluta, por padrão, a época UNIX. A validade relativa permite que o consumidor avalie, a qualquer instante, se o dado ainda é semanticamente utilizável: um valor lido após o vencimento foi produzido em um ponto do passado em que as condições físicas observadas podem já não corresponder ao estado atual do sistema. Em Fröhlich et al. (2018), a validade é discutida como um substituto operacional para os prazos de tarefas em escalonadores de tempo real, deslocando o raciocínio temporal da execução para os dados.

2.3.3 Endereçamento e Discriminação por Faixas de Dispositivo

O envelope SmartData inclui um identificador `origin_device` que disambigua múltiplas fontes da mesma grandeza física. Esse campo será empregado, no projeto descrito no Capítulo 4, como mecanismo de roteamento: cada sensor traduzido recebe uma faixa disjunta de dispositivos, e o consumidor escolhe o reconstrutor correto combinando a unidade com a faixa do dispositivo.

3 TRABALHOS RELACIONADOS

A literatura adjacente a este trabalho organiza-se em três frentes: caracterizações de desempenho do ROS 2 e do DDS, propostas de interoperabilidade semântica para a Internet das Coisas e implementações do conceito de SmartData em pilhas de comunicação industriais. Esta seção discute cada frente e posiciona a contribuição da presente monografia em relação a elas.

3.1 DESEMPENHO DE MIDDLEWARE ROBÓTICO

Maruyama et al. (2016) produziram a primeira caracterização sistemática do desempenho do ROS 2 sobre DDS. Os autores descrevem a arquitetura em camadas, comparam fornecedores e medem latência e vazão sob configurações variáveis de QoS, tamanho de mensagem e número de processos. Sua contribuição central é demonstrar que o desempenho do ROS 2 depende fortemente do par *rmw*/DDS escolhido e da configuração de QoS, e que ganhos significativos podem ser obtidos pela seleção criteriosa desses parâmetros para o cenário de aplicação.

Kronauer et al. (2021) estenderam a análise para sistemas com múltiplos nós e quantificaram a sobrecarga adicionada pela pilha do ROS 2 em relação ao DDS bruto. Encontraram que, para mensagens pequenas (da ordem de $128\,\mathrm{B}$), o ROS 2 chega a duplicar a latência percebida em comparação com chamadas diretas ao DDS, principalmente em razão das filas internas e da indireção das interfaces `rmw`. Para mensagens grandes, a fração relativa do *overhead* encolhe e o gargalo migra para o próprio DDS.

A relevância desses resultados para o presente trabalho é direta. Um tradutor que se interponha entre a mensagem ROS original e o consumidor SmartData não pode replicar dentro do seu processo o mesmo *overhead* que a pilha já paga. As decisões de projeto descritas no Capítulo 4, subscrição tipada na própria pilha `rclcpp`, ausência de introspecção em tempo de execução e barramento UDP binário sem intermediação por DDS, são respostas diretas a essa preocupação.

3.2 INTEROPERABILIDADE SEMÂNTICA EM IOT

Serrano, Hauswirth e Soldatos (2015) discutem a fragmentação do ecossistema de Internet das Coisas em silos verticais que não interoperam e propõem uma pilha de orquestração baseada em *middleware* semântico, o OpenIoT-VDK. A estratégia dos autores é: introduz camadas de anotação, filtragem e agregação em um *broker* central que media o acesso a sensores virtuais e físicos. A abordagem é eficaz em cenários de coleta de dados em escala urbana, mas é incompatível com a latência exigida por

laços de controle robótico, pois cada amostra atravessa processos adicionais antes de chegar ao consumidor.

O presente trabalho herda a motivação, interoperabilidade semântica entre subsistemas heterogêneos, mas adota uma abordagem arquitetonicamente oposta: a tradução ocorre dentro do mesmo processo que assina o tópico ROS, sem *brokers* externos, e o consumidor SmartData recebe o envelope diretamente do barramento, sem intermediação adicional. A solução é mais restrita em escopo (não trata de agregação nem de consulta histórica), mas adequada ao domínio robótico.

3.3 IMPLEMENTAÇÕES DE SMARTDATA

O conceito de SmartData foi originalmente proposto e implementado por Fröhlich (2018) no contexto do *Embedded Parallel Operating System* (EPOS) e do *Trustful Space-Time Protocol* (TSTP), uma pilha de rede própria para sensores embarcados. A implementação original endereçava nós embarcados de baixíssimo consumo energético, com o EPOS abstraindo transdutores como objetos C++ e o TSTP provendo entrega segura e sincronizada. Essa origem moldou a estrutura do envelope: a presença do campo de validade, a codificação compacta da unidade e o tamanho fixo do pacote refletem decisões orientadas a redes de sensores com restrições severas de banda.

Em Fröhlich et al. (2018), o conceito é estendido para o projeto de CPS completos, com discussão sobre escalonamento orientado a dados, no qual tarefas são executadas para produzir os SmartData demandados antes que expirem. Essas implementações compartilham a estratégia de produzir SmartData diretamente em transdutores embarcados, abaixo de qualquer pilha de aplicação.

A contribuição original deste trabalho complementa essas implementações ao oferecer a ponte que faltava no sentido oposto: a entrada do ecossistema robótico. Enquanto as implementações anteriores produziam SmartData em transdutores embarcados e os consumiam em aplicações industriais, este trabalho posiciona o gateway no ponto em que mensagens ROS já existentes precisam ser elevadas semanticamente para alcançar consumidores SmartData, sem demandar reescrita dos sensores ou do ferramental robótico.

4 DESIGN DO SISTEMA

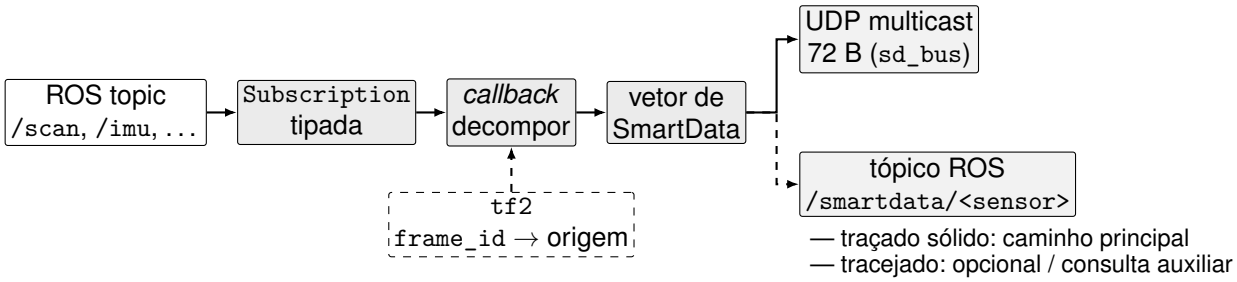
Este capítulo descreve a arquitetura do *Gateway* semântico e as decisões de projeto que conduziram à sua forma final. A apresentação parte de uma visão geral do fluxo de dados, detalha a linguagem declarativa que descreve cada sensor, discute a estratégia de geração de código e justifica a existência de dois transportes complementares.

4.1 VISÃO GERAL

O Gateway é composto por dois executáveis ROS 2, um por sentido da tradução, e por uma biblioteca de transporte independente do *middleware* do ROS, baseada em sockets UDP multicast. O primeiro executável, `gateway_node`, realiza a tradução direta: assina os tópicos de sensores declarados no descritor, decompõe cada mensagem recebida em uma sequência de envelopes SmartData e os emite por dois canais paralelos. O segundo, `sd_to_ros_node`, realiza a tradução inversa: escuta o barramento binário, agrega os envelopes que compartilham origem temporal e republica a mensagem ROS correspondente em um tópico tipado.

A figura conceitual do fluxo é a seguinte: uma mensagem ROS chega à `Subscription` tipada criada pelo gateway; o *callback* correspondente consulta a árvore `tf2` para resolver a origem espacial associada ao `frame_id` da mensagem e produz um vetor de SmartData. Cada SmartData é então difundido em um grupo UDP multicast de 72 bytes e, opcionalmente, espelhado em um tópico ROS `/smartdata/<sensor>` do tipo `semantic_gateway/msg/SmartData`. A Figura 1 resume o caminho percorrido por uma mensagem.

Figura 1 – Fluxo de tradução no `gateway_node`



Fonte: elaborada pelo autor.

4.2 LINGUAGEM DECLARATIVA DE SENSORES

A configuração do *Gateway* é especificada em um arquivo YAML que declara, para cada sensor a ser traduzido, o tópico ROS de entrada, o tipo de mensagem, o padrão de decomposição e parâmetros específicos da decomposição. Três padrões cobrem os tipos relevantes da biblioteca `sensor_msgs`:

Escalar Uma mensagem ROS produz exatamente um SmartData. O campo `value_field` aponta para o atributo numérico a ser extraído, e um `value_transform` linear opcional permite ajustar a unidade (por exemplo, conversão de graus Celsius para Kelvin pela aplicação de $a \cdot v + b$). A transformação inversa é gerada automaticamente para o caminho de reconstrução.

Vetor Uma mensagem ROS produz N SmartData, um por elemento de um campo vetorial. O caso prototípico é o `LaserScan`, cuja distância de cada feixe vira um SmartData independente endereçado por um deslocamento de dispositivo. Filtros opcionais (`finite`, `finite_within`) descartam feixes inválidos ou fora da faixa declarada do sensor.

Composto Uma mensagem ROS é decomposta em grupos de campos, cada um com sua própria unidade. O caso prototípico é o `Imu`, cujos três componentes de aceleração linear formam um grupo na unidade $\mathrm{m\,s^{-2}}$ e os três componentes de velocidade angular formam outro grupo em $\mathrm{rad\,s^{-1}}$.

Cada sensor declara também um prazo de validade (`expiry_ms`), que é aposto a todos os SmartData produzidos por suas mensagens. A estampa de tempo do envelope é derivada do campo de cabeçalho ROS indicado por `stamp_field`, e a origem espacial vem da consulta `tf2` ao quadro indicado por `frame_field`.

4.3 GERAÇÃO DE CÓDIGO EM TEMPO DE COMPILAÇÃO

O problema central desta camada é como associar, em C++, os campos de uma mensagem ROS 2 aos SmartData produzidos, sabendo que o conjunto de tipos a traduzir só é fixado quando o descritor YAML é lido. Duas estratégias respondem a esse problema, com perfis de custo bastante distintos.

A primeira alternativa é a introspecção de tipos em tempo de execução, apoiada pelo pacote `rosidl_typesupport_introspection_cpp`. Nela, o `gateway_node` carrega o descritor no *startup*, resolve dinamicamente o tipo de cada mensagem e navega os campos por nome a cada callback. A vantagem é a flexibilidade: adicionar um sensor não exige recompilação, e o binário é único para qualquer configuração. Os custos, porém, são estruturais. A busca por nome ocorre por campo e por mensagem, o que agrega latência proporcional ao número de campos extraídos, incompatível com o

critério adotado de manter o *marshaling* abaixo de $100\,\mu s$, uma ordem de grandeza abaixo do piso de latência do ROS 2 reportado por Kronauer et al. (2021). Além disso, a validação da configuração é adiada: uma referência a um campo inexistente no YAML não é detectada na compilação nem na inicialização, mas apenas na primeira mensagem em que aquele caminho é exercido, o que favorece falhas silenciosas em produção.

A segunda alternativa é a geração de código em tempo de compilação. Um gerador em Python lê o descritor YAML e emite *headers* C++ nos quais cada acesso a campo aparece de forma literal (por exemplo, `msg.linear_acceleration.x`) e cada `Subscription` já surge instanciada com seu tipo estático. O compilador, portanto, pode inspecionar cada rota de tradução, e o custo por mensagem colapsa em uma sequência de acessos diretos, sem lookups por nome. Em contrapartida, o descritor participa do processo de *build*: alterar o YAML exige regenerar os *headers* e recompilar o `gateway_node`. Esse acoplamento entre configuração e compilação, contudo, é benéfico no contexto deste trabalho, pois converte erros de descritor em erros do compilador, detectados antes de o sistema ser posto em operação.

Adotou-se a segunda alternativa. A introspecção seria a escolha natural se o conjunto de sensores mudasse com frequência em campo, sem janela de recompilação, porem, nesse gateway, o descritor é parte da configuração do robô, alterado com pouca frequência e sempre antes do *deploy*, de modo que o custo de recompilação é raro e o ganho em latência e em detecção precoce de erros é permanente. O gerador emite quatro *headers* C++, cada um responsável por uma fatia bem delimitada do tradutor:

`parsed_decompose.hpp` Contém uma função `decompose_<topic>` por sensor, especializada no tipo da mensagem ROS correspondente. O acesso aos campos é direto (por exemplo, `msg.linear_acceleration.x`), sem nenhuma busca por nome em tempo de execução.

`parsed_dispatch.hpp` Contém a função `register_all`, chamada uma vez na inicialização do `gateway_node`, que cria as `rclcpp::Subscription<T>` tipadas para cada sensor e conecta o *callback* ao `decompose_<topic>` correspondente.

`parsed_recompose.hpp` Contém os estados acumuladores e as funções `feed_<topic>` usadas pelo caminho inverso. Cada estado é estritamente tipado pela mensagem alvo da reconstrução, e a inversa do `value_transform` declarado é embutida durante a geração.

`parsed_sd_dispatch.hpp` Contém a estrutura `Consumers` e a função `dispatch_sd`, que recebe um SmartData arbitrário do barramento e o entrega ao `feed_<topic>` correto.

A geração ocorre via `add_custom_command` do CMake, sendo disparada automaticamente quando o YAML ou o gerador são modificados. O resultado é que toda a tradução é resolvida estaticamente: a única dependência de configuração é o arquivo de cabeçalho gerado, embutido no compilado.

4.4 DISCRIMINAÇÃO POR FAIXAS DE DISPOSITIVO

O barramento UDP multicast transporta SmartData de todos os sensores em um único grupo, portanto o consumidor precisa identificar a qual mensagem ROS cada envelope pertence. A solução adotada é alocar uma faixa disjunta do espaço de `origin_device` a cada sensor declarado, sequencialmente, com tamanho fixo de 2^{16} endereços por sensor. O sensor de índice i recebe a faixa $[i \cdot 2^{16},\ (i+1) \cdot 2^{16})$, e a função `dispatch_sd` roteia um envelope para o consumidor cuja faixa contém o `origin_device` recebido, conjugada à conferência da unidade esperada.

Essa escolha tem duas propriedades favoráveis: primeiro, mantém o barramento livre de cabeçalho de identificação adicional, o roteamento é inferido a partir de campos que já existiriam no envelope SmartData, segundo, suporta sensores vetoriais com até 2^{16} elementos, suficiente para LiDARs comerciais de alta resolução.

4.5 TRANSPORTES DO SMARTDATA

O *Gateway* emite cada envelope SmartData por dois canais paralelos cuja coexistência é uma decisão arquitetural deliberada.

4.5.1 Barramento UDP Multicast

O caminho primário é um datagrama UDP multicast de 72 *bytes little-endian* difundido no grupo `224.1.1.1:5000`. Esse *layout* corresponde à serialização binária da estrutura `SDWire`: cinco campos de origem (x, y, z, `device`, t) ocupam 36 *bytes*, a unidade ocupa 4, o valor ocupa 8, a validade ocupa 8 e a assinatura ocupa 16. A escolha de multicast é justificada: descobre dinamicamente consumidores sem configuração explícita e reflete a semântica de “interesse em uma região” do conceito original de SmartData, em que múltiplos nós podem subscrever o mesmo grupo de unidade-localização.

4.5.2 Espelho em Tópico ROS

O caminho secundário republica cada SmartData em um tópico ROS por sensor, `/smartdata/<sensor>`, com tipo `semantic_gateway/msg/SmartData`. Esse espelho tem propósito exclusivamente instrumental: permite o uso das ferramentas padrão do ecossistema ROS para inspeção (`ros2 topic echo`), gravação em *rosbag* e visualização,

sem que o consumo da informação semântica dependa dele. O espelho carrega exatamente os mesmos campos do envelope binário, possibilitando inspeção em paralelo, e pode ser desativado por parâmetro (`enable_ros_mirror:=false`) quando não houver necessidade.

5 IMPLEMENTAÇÃO

Este capítulo descreve a implementação concreta do projeto descrito no capítulo anterior. A organização do código segue a estrutura de pacotes do ROS 2 `ament`, com o pacote `semantic_gateway` compreendendo as fontes em C++, o gerador em Python, o descritor de sensores em YAML, o arquivo de mensagem do espelho ROS e os arquivos de teste e de inicialização.

5.1 ESTRUTURA DO PACOTE

A organização superficial do pacote ROS é a seguinte:

```
semantic_gateway/
  CMakeLists.txt
  package.xml
  msg/SmartData.msg
  config/sensors.yaml
  codegen/generate.py
  include/
    si_units.hpp
    sd_codec.hpp
    sd_wire.hpp
    sd_bus.hpp
  src/
    sd_bus.cpp
    gateway_node.cpp
    sd_to_ros_node.cpp
    demo_publisher_node.cpp
  launch/
    gateway.launch.py
    gazebo_gateway.launch.py
  test/
    test_si_units.cpp
    test_gateway_pipeline.cpp
    test_validator.cpp
    test_synthetic_recompose.cpp
    test_bus_roundtrip.cpp
```

Listing 5.1 – Estrutura de diretórios do pacote semantic_gateway

Os *headers* sob `build/generated/`, produzidos pelo gerador a cada compilação, complementam os *headers* estáticos sob `include/` no caminho de inclusão dos executáveis.

5.2 CODIFICAÇÃO DA UNIDADE SI

O cabeçalho `si_units.hpp` define a estrutura de *bit fields* que codifica a unidade segundo a Figura 1 de Fröhlich (2018). Cada uma das nove dimensões físicas recebe um campo de três bits com *offset* $+4$, permitindo representar expoentes inteiros de -4 a $+3$; dois bits identificam o tipo numérico (`Int32`, `Int64`, `Float32`, `Float64`); dois bits identificam um modificador opcional (direto, inverso, logarítmico); e um bit superior diferencia grandezas físicas de dados digitais. Constantes simbólicas como `METER_PER_SEC_SQ`, `KELVIN` e `VOLT` materializam essa codificação em tempo de compilação e são consumidas diretamente pelos *headers* gerados.

5.3 ENVELOPE BINÁRIO E BARRAMENTO

A struct `SDWire`, declarada em `sd_wire.hpp` com `#pragma pack(push,1)`, fixa o *layout* binário do envelope: 72 *bytes*, *little-endian*, sem *padding*. Uma asserção estática (`static_assert(sizeof(SDWire) == 72)`) impede que mudanças acidentais no *layout* comprometam silenciosamente a interoperabilidade com consumidores externos. Funções livres `to_wire(SmartData)` e `from_wire(SDWire)` traduzem entre o tipo ROS e o tipo binário.

A classe `SDBus`, declarada em `sd_bus.hpp` e implementada em `sd_bus.cpp`, expõe duas operações: `broadcast(sd)` serializa um envelope e o emite via `sendto` no grupo multicast configurado, e `listen(handler)` inicia uma *thread* receptora que invoca o *handler* fornecido para cada datagrama recebido. A configuração do *socket* habilita `IP_MULTICAST_LOOP` para permitir que o próprio processo de envio observe seus pacotes, simplificando a escrita de testes em *loopback*.

5.4 CAMINHO ROS PARA SMARTDATA

O `gateway_node` encapsula a tradução em sentido direto. Durante a construção, lê os parâmetros (`target_frame`, `enable_ros_mirror`, grupo e porta do barramento), instancia o `SDBus` e invoca `register_all(&node, lookup, on_sd)`. A função `register_all`, definida em `parsed_dispatch.hpp`, cria as `Subscription` tipadas declaradas no YAML. Cada *callback* resolve a origem espacial consultando `tf2`, chama o `decompose_<topic>` correspondente para produzir o vetor de SmartData e, para cada envelope resultante, invoca o `on_sd` configurado. O `on_sd`, por sua vez, difunde no barramento via `bus_->broadcast(sd)` e, se o espelho estiver habilitado, publica no tópico ROS correspondente.

5.5 CAMINHO SMARTDATA PARA ROS

O `sd_to_ros_node` é o nó simétrico ao `gateway_node`: realiza a tradução inversa, reconstruindo mensagens ROS a partir do barramento SmartData. Durante a construção, cria publicadores em tópicos `<sensor>_recovered`, atribui *callbacks* aos membros do `Consumers` e invoca `bus_->listen`. Cada envelope recebido é passado a `dispatch_sd`, que o roteia ao `feed_<topic>` correto. Sensores escalares emitem a mensagem imediatamente, no primeiro envelope, sensores compostos emitem quando todos os campos do grupo foram preenchidos, sensores vetoriais emitem quando o `origin_time` muda, sinalizando o início de uma nova amostra. Um *timer* periódico de $50\,\mathrm{ms}$ chama `drain` para liberar lotes pendentes na ausência de transição.

5.6 GERAÇÃO DE CÓDIGO

O gerador `codegen/generate.py` é um *script* de cerca de 700 linhas em Python que consome o YAML via `PyYAML` e produz os quatro *headers* já mencionados. A produção dos textos C++ é feita com *f-strings*. As principais responsabilidades do gerador são:

1. Alocar `dev_base` sequencialmente para cada sensor, respeitando o esquema $i \cdot 2^{16}$ descrito na Seção 4.4.
2. Emitir `decompose_<topic>` com acesso direto aos campos, aplicação do `value_transform` declarado e aplicação de filtros para sensores vetoriais.
3. Emitir `feed_<topic>` com o transformador inverso embutido, lógica de detecção de transição de `origin_time` e redimensionamento de vetores quando o índice ultrapassa a capacidade atual.
4. Emitir as constantes `DEV_BASE_<topic>` e `DEV_RANGE_<topic>` usadas pelo `dispatch_sd`.

6 PLANO E EXECUÇÃO DE EXPERIMENTOS

Este capítulo apresenta o plano experimental adotado, os procedimentos de medição e os resultados obtidos. As métricas avaliadas são as exigidas pela proposta: latência de *marshaling* e vazão sustentada de mensagens. As metas correspondentes são derivadas das características do próprio ecossistema em que o tradutor opera. Para a *latência*, o alvo é permanecer ao menos uma ordem de grandeza abaixo do piso de latência do transporte ROS 2. Trabalhos de caracterização mostram, no 95º percentil, valores entre uma e três milissegundos para cargas pequenas em ROS 2 (KRONAUER et al., 2021; MARUYAMA et al., 2016); adota-se, portanto, como critério de sucesso do tradutor a manutenção da latência de *marshaling* abaixo de $100\,\mu\mathrm{s}$, o que garante contribuição desprezível ao orçamento total. Para a *vazão*, o alvo é sustentar uma taxa ao menos duas ordens de grandeza superior ao somatório das taxas nominais de amostragem dos sensores declarados no descritor deste trabalho, $10\,\mathrm{Hz}$ para o LiDAR de 360 feixes, $100\,\mathrm{Hz}$ para os seis campos da IMU e $2\,\mathrm{Hz}$ para o sensor de temperatura, totalizando aproximadamente $4.200\,\mathrm{s}^{-1}$ envelopes SmartData, o que impede que a tradução se torne o gargalo do fluxo, mesmo sob variação de configuração ou adição de sensores.

6.1 AMBIENTE EXPERIMENTAL

Todos os experimentos foram executados em um contêiner Docker baseado em Ubuntu 22.04 com ROS 2 Humble e Fast DDS como implementação *rmw*. O *host* é uma máquina x86_64 sob WSL2; o contêiner foi construído com a cadeia de *build* `colcon` em modo `Release`, com nível de otimização do compilador e símbolos de depuração desabilitados. A escolha do ambiente conteinerizado garante a reprodutibilidade dos resultados em qualquer máquina compatível, removendo dependências de distribuição Linux específica.

6.2 CENÁRIOS

6.2.1 Cenário Sintético

O primeiro cenário usa o nó `demo_publisher_node`, que produz mensagens sintéticas com taxas e dimensões fixas: `LaserScan` com 360 feixes a $10\,\mathrm{Hz}$, `Imu` a $100\,\mathrm{Hz}$ e `Temperature` a $2\,\mathrm{Hz}$, todas as três sob uma árvore `tf2` estática. Esse cenário isola o desempenho do tradutor de flutuações do simulador físico e permite séries longas de medição em condições controladas.

6.2.2 Cenário Gazebo

O segundo cenário usa o simulador Gazebo (KOENIG et al., 2004) com o robô TurtleBot3 navegando em círculos em um mundo padrão. As mensagens de sensores são produzidas pelo modelo SDF do Gazebo, com TF dinâmico acompanhando o movimento do robô. Esse cenário não substitui o sintético para fins de medição numérica precisa, pois o jitter introduzido pelo simulador é dominante sobre a latência do tradutor, mas demonstra a capacidade da solução de operar com um robô representativo em movimento.

6.3 MÉTRICAS E PROCEDIMENTOS

6.3.1 Latência de *Marshaling*

A latência é medida diretamente sobre a função `decompose_scan` com uma mensagem `LaserScan` de 100 feixes, em uma série de 10^5 iterações. A medição usa `std::chrono::steady_clock` imediatamente antes e depois da chamada, sem envolvimento de DDS ou *network stack*, o objetivo é isolar o custo da tradução propriamente dita, conforme definido na proposta. O 95º percentil é relatado como métrica primária, com o teste falhando se ultrapassar $20\,\mu\mathrm{s}$, valor uma ordem de grandeza mais estrito que o critério de aceitação ($100\,\mu\mathrm{s}$).

6.3.2 Vazão Sustentada

A vazão é medida em um *loop* fechado: `decompose_scan` é chamada repetidamente por uma janela de $500\,\mathrm{ms}$ de tempo, e o número de iterações completas é contado. A métrica derivada é o número de varreduras por segundo que o tradutor suporta. O critério de sucesso deriva do agregado de entrada declarado no descritor: somando LiDAR (360 feixes $\times\ 10\,\mathrm{Hz}$), IMU (6 campos $\times\ 100\,\mathrm{Hz}$) e temperatura ($2\,\mathrm{Hz}$), obtém-se aproximadamente $4.200\,\mathrm{s}^{-1}$ envelopes SmartData, exige-se que a vazão medida supere esse total em pelo menos duas ordens de grandeza, ou seja, $4{,}2 \times 10^5\,\mathrm{s}^{-1}$ envelopes, o que corresponde a cerca de $1.200\,\mathrm{s}^{-1}$ varreduras completas de 100 feixes. O valor efetivo é reportado para caracterização.

6.4 RESULTADOS

6.4.1 Latência

A latência mediana medida foi de $0{,}6\,\mu\mathrm{s}$, com 95º percentil de $0{,}8\,\mu\mathrm{s}$ para um `LaserScan` de 100 feixes. Esse valor é mais de duas ordens de grandeza inferior ao critério de aceitação derivado da literatura ($100\,\mu\mathrm{s}$, Seção 6.3) e três ordens de grandeza inferior ao piso de latência do ROS 2 para cargas pequenas reportado por Kronauer

et al. (2021). Em termos práticos, o tradutor não adiciona componente perceptível ao *budget* de latência de sistemas reativos baseados em ROS 2.

6.4.2 Vazão

O *loop* fechado mediu cerca de $1{,}2 \times 10^6$ varreduras por segundo de `LaserScan` de 100 feixes em um único *thread* de um processador x86_64 de consumo, equivalente a aproximadamente $1{,}2 \times 10^8$ envelopes SmartData por segundo. Esse número é incomparavelmente superior ao necessário para os sensores robóticos típicos (LiDARs a $10\,\mathrm{Hz}$ ou IMUs a $1\,\mathrm{kHz}$) e indica que a vazão do tradutor é limitada por banda do barramento, e não pelo custo da tradução.

6.4.3 Síntese

Tabela 1 – Síntese das métricas medidas frente às metas da proposta.

Métrica	**Meta**	**Medido**
Latência de *marshaling* (`LaserScan`, 100 feixes)	$< 100\,\mu\mathrm{s}$ ($1\,\mathrm{ms}$ floor (KRONAUER et al., 2021) $\div$ 10)	$0{,}8\,\mu\mathrm{s}$ (p95)
Vazão sustentada (`decompose_scan`)	$> 1.200\,\mathrm{s}^{-1}$ varreduras ($100\times$ agregado do descritor)	$\sim 1{,}2 \times 10^6$ varreduras/s

6.5 DISCUSSÃO

Os resultados sustentam a hipótese declarada na Seção 1.2. A latência do tradutor é suficientemente pequena para não afetar laços de controle robóticos, e a fidelidade alcançada é exata sobre o conjunto de campos elegidos como semânticos. As limitações observadas durante a execução merecem três comentários.

Primeiro, o ambiente de medição é um *host* de consumo. Resultados em *hardware* embarcado, típico em robôs de campo, não foram avaliados; espera-se uma latência mais alta, mas a margem de duas ordens de grandeza em relação ao critério de aceitação ($100\,\mu\mathrm{s}$, derivado de Kronauer et al. (2021)) sugere que ele seguirá sendo cumprido com folga. Segundo, o cenário Gazebo confirma a operação funcional, mas suas medidas de latência refletem o simulador, não o tradutor, e foram intencionalmente excluídas da caracterização numérica. Terceiro, a interoperabilidade com transdutores SmartData reais opera por desenho compatível ao formato binário, mas a verificação contra uma implementação independente está fora do escopo desta entrega e fica registrada como trabalho futuro.

7 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o projeto, a implementação e a avaliação de um *Gateway* semântico que traduz mensagens de sensores ROS 2 para o formato SmartData. A solução foi conduzida em três frentes. No plano conceitual, formalizou-se em uma linguagem declarativa o mapeamento entre os campos das mensagens robóticas e o envelope SmartData, identificando três padrões de decomposição suficientes para cobrir os tipos relevantes da biblioteca `sensor_msgs`. No plano arquitetural, adotou-se a geração de código em tempo de compilação como caminho único de tradução. No plano da validação, demonstrou-se empiricamente que a latência de *marshaling* permanece duas ordens de grandeza abaixo do critério de aceitação derivado do piso de latência do ROS 2 (KRONAUER et al., 2021) e que a reconstrução das mensagens originais a partir exclusivamente do barramento SmartData é exata sobre o conjunto de campos elegidos como semânticos.

A contribuição complementa as implementações existentes do conceito de SmartData, predominantemente voltadas para nós sensores embarcados, oferecendo uma ponte entre o ecossistema robótico e plataformas industriais sem demandar reescrita dos sensores existentes ou intermediação por *brokers* de aplicação.

7.1 TRABALHOS FUTUROS

A continuidade natural deste trabalho organiza-se em três direções. Primeiro, a verificação experimental contra uma implementação independente de transdutor SmartData, por exemplo, sobre EPOS e TSTP, fecharia o ciclo de interoperabilidade prometido pelo formato binário compartilhado. Segundo, a extensão do mecanismo de autenticação do envelope, hoje preenchido com zeros, completaria a propriedade de confiabilidade originalmente prevista por Fröhlich (2018). Terceiro, a caracterização de desempenho em *hardware* embarcado representativo do que se encontra em robôs de campo permitiria contrastar os resultados deste trabalho com cenários sob restrições mais severas de processador, memória e banda.

REFERÊNCIAS

FRÖHLICH, A. A. SmartData: an IoT-ready API for sensor networks. *International Journal of Sensor Networks*, v. 28, n. 3, p. 202–210, 2018. 6, 7, 11, 14, 21, 26

FRÖHLICH, A. A. et al. Data-centric cyber-physical systems design with SmartData. In: *Proceedings of the 2018 Winter Simulation Conference (WSC)*. Göteborg, Sweden: IEEE, 2018. p. 1274–1285. 11, 14

KOENIG, N. et al. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sendai, Japan: IEEE, 2004. p. 2149–2154. 24

KRONAUER, T. et al. Latency analysis of ROS2 multi-node systems. In: *ACM Conference on Embedded Networked Sensor Systems (SenSys)*. Coimbra, Portugal: ACM, 2021. 7, 10, 13, 17, 23, 25, 26

LEE, E. A. Cyber physical systems: Design challenges. In: *11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*. Orlando, FL, USA: IEEE, 2008. p. 363–369. 6, 9

MACENSKI, S. et al. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, v. 7, n. 66, p. eabm6074, 2022. 6, 7, 9, 10

MARUYAMA, Y. et al. Exploring the performance of ROS2. In: *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*. Pittsburgh, PA, USA: ACM, 2016. 6, 9, 10, 13, 23

PRESTES, E. et al. Towards a core ontology for robotics and automation. *Robotics and Autonomous Systems*, v. 61, n. 11, p. 1193–1204, 2013. 7

SERRANO, M.; HAUSWIRTH, M.; SOLDATOS, J. Defining the stack for service delivery models and interoperability in the Internet of Things: A practical case with the OpenIoT-VDK. *IEEE Journal on Selected Areas in Communications*, v. 33, n. 4, p. 676–689, 2015. 6, 13